

A Practical Primer on Quadratic Programming

From First Principles and Geometry to Multi-Obstacle Drone Safety Filters

Autonomous Aerial Systems Course Notes

September 22, 2026

Abstract

Quadratic Programming (QP) represents the cornerstone of modern computational optimal control, robotic motion planning, and real-time safety filtering. This tutorial provides a rigorous, accessible introduction to Quadratic Programming tailored for robotics engineers and autonomy practitioners. We begin with a concrete 2-dimensional unconstrained QP, deriving the global minimizer analytically through explicit partial differentiation and matrix calculus. Next, we examine the geometric mechanism through which linear equality and inequality constraints reshape the solution, illuminating the interplay between expanding level sets, active boundaries, and the Karush-Kuhn-Tucker (KKT) stationarity conditions. We then present the canonical QP standard form and demonstrate how to formulate and solve constrained problems in Python using the interior-point solver `CVXOPT`. Finally, we bridge mathematical theory to aerial robotics by applying QP to the **Multi-Obstacle Control Barrier Function (CBF)** safety filter for autonomous quadrotors, explaining why multiple simultaneous obstacles necessitate online numerical QP solvers and demonstrating real-time implementation.

Contents

1	Introduction: The Centrality of Quadratic Programming in Robotics	2
2	A 2-Dimensional Unconstrained Quadratic Program	2
2.1	Scalar and Algebraic Definition	3
2.2	Derivation via Explicit Partial Differentiation	3
2.3	Matrix Representation and Gradient Derivation	3
2.4	Second-Order Sufficiency and Convexity	4
3	Conceptual Geometry: How Constraints Reshape the Solution	4
3.1	Geometry of Unconstrained Level Sets	4
3.2	Introducing Linear Inequality Constraints	5
3.3	Active vs. Inactive Constraints	6
3.4	The KKT Conditions: Geometric Force Balance	6
4	The General Quadratic Program and the CVXOPT Solver	7
4.1	Canonical Standard Formulation	7
4.2	Overview of the CVXOPT Solver	8
4.3	Solving QPs with <code>cvxopt.solvers.qp</code>	8
4.3.1	Important Usage Guidelines	8
4.4	Self-Contained Python Example	9
5	Robotics Application: Multi-Obstacle Control Barrier Functions	10
5.1	System Architecture and Single-Integrator Kinematics	10
5.2	Nominal Guidance: Defining $\mathbf{v}_{\text{des}}$ via a Position PID Controller	10
5.2.1	Continuous-Time Position PID Formulation	10
5.2.2	Integral Anti-Windup Clamping and Speed Saturation	11

5.2.3	Discrete-Time Implementation (50 Hz Control Loop)	11
5.2.4	Why the PID Command Must Be Filtered by a CBF	11
5.3	Quantifying Safety: The Barrier Margin $h_i(\mathbf{p})$	12
5.4	Forward Set Invariance and Nagumo's Condition	12
5.5	Why Multiple Obstacles Demand a QP Solver	13
5.6	Formulating the Multi-Obstacle CBF-QP	14
5.7	Mapping to CVXOPT Standard Matrices	14
5.8	Python Implementation: <code>MultiObstacleCBFSafetyFilter</code>	15
5.9	Numerical Case Study	16
6	Summary and Practical Takeaways	17

1 Introduction: The Centrality of Quadratic Programming in Robotics

In autonomous robotics, decision-making algorithms must compute control actions that balance conflicting demands: tracking high-level mission goals, respecting physical actuator saturation, and strictly preventing catastrophic collisions. Under real-time execution budgets ranging from 10 Hz to 1000 Hz, optimization algorithms must be computationally predictable, globally convergent, and robust against local minima.

Among the broad family of mathematical programming paradigms, **Quadratic Programming (QP)** occupies a uniquely privileged position:

1. **Global Convexity:** Whenever the quadratic cost matrix $\mathbf{P}$ is positive semi-definite ($\mathbf{P} \succeq 0$), any local minimizer is guaranteed to be a global minimizer. If $\mathbf{P} \succ 0$ (strictly positive definite), the global minimizer is strictly unique.
2. **Linear Optimality Conditions:** The first-order optimality conditions (Karush-Kuhn-Tucker conditions) for a QP result in a system of linear equalities and simple complementarity inequalities.
3. **Polynomial-Time Solvability:** Primal-dual interior-point methods can solve constrained QPs with dozens or hundreds of variables and constraints in under a millisecond with deterministic polynomial-time complexity $\mathcal{O}(n^3)$.
4. **Natural Physical Formulation:** Energy, tracking errors ($\|\mathbf{x} - \mathbf{x}_{\text{des}}\|^2$), control effort ($\|\mathbf{u}\|^2$), and momentum are naturally quadratic, while kinematics, motor limits, and barrier conditions are frequently linear in the control variables.

Whether in Model Predictive Control (MPC), operational space inverse kinematics, legged robot contact force distribution, or Control Barrier Function (CBF) safety filters, the ability to formulate and solve QPs is an essential skill for roboticists.

2 A 2-Dimensional Unconstrained Quadratic Program

To build intuition, we begin with an unconstrained optimization problem in $\mathbb{R}^2$, deriving its solution through direct scalar calculus and then generalizing to matrix notation.

2.1 Scalar and Algebraic Definition

Consider an unconstrained scalar objective function $f(x_1, x_2) : \mathbb{R}^2 \rightarrow \mathbb{R}$ defined by:

$$f(x_1, x_2) = \frac{1}{2} (p_{11}x_1^2 + 2p_{12}x_1x_2 + p_{22}x_2^2) + q_1x_1 + q_2x_2 + r \quad (1)$$

where $p_{11}, p_{12}, p_{22}, q_1, q_2, r \in \mathbb{R}$ are constant parameters.

2.2 Derivation via Explicit Partial Differentiation

To find the stationary point (x_1^*, x_2^*) that minimizes f , Fermat's theorem requires all first-order partial derivatives to vanish simultaneously:

$$\frac{\partial f}{\partial x_1} = p_{11}x_1 + p_{12}x_2 + q_1 = 0 \quad (2)$$

$$\frac{\partial f}{\partial x_2} = p_{12}x_1 + p_{22}x_2 + q_2 = 0 \quad (3)$$

Equations (2) and (3) form a linear system of two equations in two unknowns:

$$\begin{bmatrix} p_{11} & p_{12} \\ p_{12} & p_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (4)$$

Using Cramer's rule or 2D matrix inversion, let $\Delta = p_{11}p_{22} - p_{12}^2$ denote the determinant. If $\Delta > 0$ and $p_{11} > 0$, the unique solution is:

$$x_1^* = \frac{-q_1p_{22} + q_2p_{12}}{p_{11}p_{22} - p_{12}^2} \quad (5)$$

$$x_2^* = \frac{-q_2p_{11} + q_1p_{12}}{p_{11}p_{22} - p_{12}^2} \quad (6)$$

2.3 Matrix Representation and Gradient Derivation

We can express Equation (1) compactly in vector-matrix form. Define the decision vector $\mathbf{x}$, the symmetric Hessian matrix $\mathbf{P}$, the linear cost vector $\mathbf{q}$, and scalar offset r :

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^2, \quad \mathbf{P} = \begin{bmatrix} p_{11} & p_{12} \\ p_{12} & p_{22} \end{bmatrix} \in \mathbb{S}^2, \quad \mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} \in \mathbb{R}^2, \quad r \in \mathbb{R} \quad (7)$$

The quadratic function becomes:

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{P} \mathbf{x} + \mathbf{q}^T \mathbf{x} + r \quad (8)$$

Mathematical Definition: Vector Gradient and Hessian of a Quadratic Form

For a symmetric matrix $\mathbf{P} = \mathbf{P}^T$, the gradient vector $\nabla f(\mathbf{x}) \in \mathbb{R}^n$ and Hessian matrix $\nabla^2 f(\mathbf{x}) \in \mathbb{R}^{n \times n}$ are:

$$\nabla f(\mathbf{x}) = \mathbf{P} \mathbf{x} + \mathbf{q} \quad (9)$$

$$\nabla^2 f(\mathbf{x}) = \mathbf{P} \quad (10)$$

Setting the vector gradient to zero yields the celebrated linear normal equation:

$$\nabla f(\mathbf{x}^*) = \mathbf{P} \mathbf{x}^* + \mathbf{q} = \mathbf{0} \implies \mathbf{P} \mathbf{x}^* = -\mathbf{q} \quad (11)$$

If $\mathbf{P}$ is invertible, the unconstrained global minimizer is given in closed form by:

$$\mathbf{x}_{\text{unc}}^* = -\mathbf{P}^{-1} \mathbf{q} \quad (12)$$

2.4 Second-Order Sufficiency and Convexity

To verify that $\mathbf{x}^*$ is a strict local (and global) minimum rather than a maximum or saddle point, we evaluate the second derivative (Hessian matrix):

$$\nabla^2 f(\mathbf{x}) = \mathbf{P} \quad (13)$$

By Sylvester's criterion, $\mathbf{P}$ is positive definite ($\mathbf{P} \succ 0$) if and only if all its leading principal minors are positive:

$$p_{11} > 0 \quad \text{and} \quad \det(\mathbf{P}) = p_{11}p_{22} - p_{12}^2 > 0 \quad (14)$$

When $\mathbf{P} \succ 0$, $f(\mathbf{x})$ is strictly convex, guaranteeing that $\mathbf{x}_{\text{unc}}^* = -\mathbf{P}^{-1}\mathbf{q}$ is the unique global minimizer. The minimum cost value attained is:

$$f(\mathbf{x}_{\text{unc}}^*) = -\frac{1}{2}\mathbf{q}^T\mathbf{P}^{-1}\mathbf{q} + r \quad (15)$$

Step-by-Step Worked Example: Concrete Numerical Walkthrough

Let us solve an unconstrained 2D problem with specific numbers:

$$f(x_1, x_2) = 2x_1^2 + x_1x_2 + x_2^2 - 6x_1 - 4x_2 + 10$$

Step 1: Identify Parameters:

$$\mathbf{P} = \begin{bmatrix} 4 & 1 \\ 1 & 2 \end{bmatrix}, \quad \mathbf{q} = \begin{bmatrix} -6 \\ -4 \end{bmatrix}, \quad r = 10$$

Step 2: Check Positive Definiteness:

$$p_{11} = 4 > 0, \quad \det(\mathbf{P}) = (4)(2) - (1)(1) = 7 > 0 \implies \mathbf{P} \succ 0 \quad (\text{Strictly Convex!})$$

Step 3: Invert $\mathbf{P}$:

$$\mathbf{P}^{-1} = \frac{1}{7} \begin{bmatrix} 2 & -1 \\ -1 & 4 \end{bmatrix}$$

Step 4: Compute Optimal Solution:

$$\mathbf{x}_{\text{unc}}^* = -\mathbf{P}^{-1}\mathbf{q} = -\frac{1}{7} \begin{bmatrix} 2 & -1 \\ -1 & 4 \end{bmatrix} \begin{bmatrix} -6 \\ -4 \end{bmatrix} = \frac{1}{7} \begin{bmatrix} 8 \\ 10 \end{bmatrix} = \begin{bmatrix} 1.143 \\ 1.429 \end{bmatrix}$$

Step 5: Minimum Value:

$$f(\mathbf{x}_{\text{unc}}^*) = 10 - \frac{1}{2} \begin{bmatrix} -6 & -4 \end{bmatrix} \frac{1}{7} \begin{bmatrix} 8 \\ 10 \end{bmatrix} = 10 - \frac{88}{14} \approx 4.143$$

3 Conceptual Geometry: How Constraints Reshape the Solution

In practical applications, variables cannot take arbitrary values. Control inputs are limited by motor voltage, quadcopter velocities are bounded by aerodynamics, and positions must avoid obstacles. We now introduce constraints and examine their geometric influence.

3.1 Geometry of Unconstrained Level Sets

The level curves (contours) of the unconstrained objective $f(\mathbf{x}) = c$ are defined by:

$$\frac{1}{2}(\mathbf{x} - \mathbf{x}_{\text{unc}}^*)^T \mathbf{P} (\mathbf{x} - \mathbf{x}_{\text{unc}}^*) = c - f(\mathbf{x}_{\text{unc}}^*) \quad (16)$$

Because $\mathbf{P} \succ 0$, these level curves form concentric **ellipses** centered at $\mathbf{x}_{\text{unc}}^*$:

- The principal axes of the ellipses align with the eigenvectors of $\mathbf{P}$.
- The lengths of the semi-axes are inversely proportional to $\sqrt{\lambda_i(\mathbf{P})}$, where λ_i are the eigenvalues of $\mathbf{P}$.
- As c increases, the ellipses expand outward like ripples in a pond away from the unconstrained bottom of the bowl.

3.2 Introducing Linear Inequality Constraints

Now consider a single linear inequality constraint defining a closed half-space:

$$g_1x_1 + g_2x_2 \leq h \iff \mathbf{g}^T \mathbf{x} \leq h \quad (17)$$

The vector $\mathbf{g} = [g_1, g_2]^T$ is the **outward normal vector** perpendicular to the boundary line $\mathbf{g}^T \mathbf{x} = h$. The constraint partitions $\mathbb{R}^2$ into two halves:

- The **Feasible Set** $\mathcal{F} = \{\mathbf{x} \in \mathbb{R}^2 \mid \mathbf{g}^T \mathbf{x} \leq h\}$.
- The **Infeasible Set** $\{\mathbf{x} \in \mathbb{R}^2 \mid \mathbf{g}^T \mathbf{x} > h\}$.

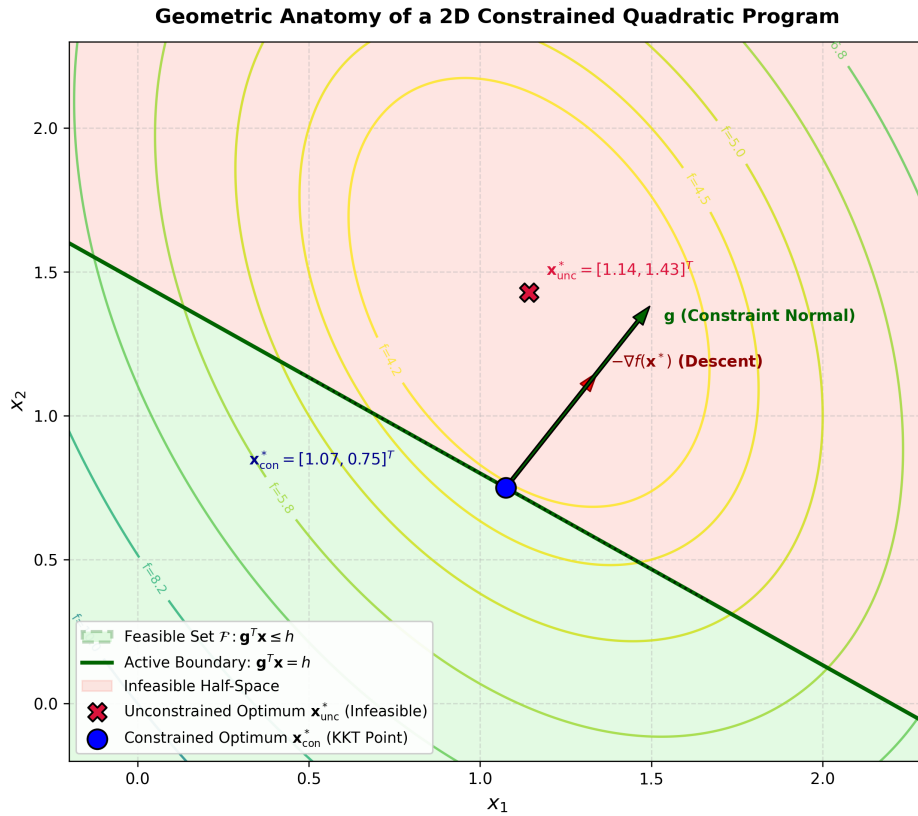


Figure 1: **Geometric Anatomy of a Constrained Quadratic Program in $\mathbb{R}^2$.** The unconstrained minimum $\mathbf{x}_{\text{unc}}^*$ lies in the infeasible half-space. The level sets of $f(\mathbf{x})$ expand outward until the first ellipse touches the feasible boundary $\mathbf{g}^T \mathbf{x} = h$ at $\mathbf{x}_{\text{con}}^*$. At this tangency point, the descent direction $-\nabla f(\mathbf{x}^*)$ points directly into the forbidden region and is perfectly balanced by the outward constraint normal $\mathbf{g}$.

3.3 Active vs. Inactive Constraints

When we solve $\min_{\mathbf{x} \in \mathcal{F}} f(\mathbf{x})$, two distinct geometric scenarios can occur:

Intuition & Geometric Insight: The Expanding Ripple Analogy

Imagine releasing a drop of water at $\mathbf{x}_{\text{unc}}^*$. The ripples expand as concentric ellipses of increasing cost $f(\mathbf{x})$:

- **Scenario A: Inactive (Slack) Constraint** ($\mathbf{g}^T \mathbf{x}_{\text{unc}}^* < h$): The unconstrained minimum $\mathbf{x}_{\text{unc}}^*$ already lies comfortably inside the feasible region $\mathcal{F}$. The constraint is **inactive** (it exerts no force on the solution). The constrained minimizer is simply:

$$\mathbf{x}_{\text{con}}^* = \mathbf{x}_{\text{unc}}^* \quad (18)$$

The Lagrange multiplier associated with this constraint is zero ($\lambda = 0$).

- **Scenario B: Active (Tight) Constraint** ($\mathbf{g}^T \mathbf{x}_{\text{unc}}^* > h$): The unconstrained ideal $\mathbf{x}_{\text{unc}}^*$ lies in the forbidden region and is physically unattainable. The expanding ripple touches the boundary $\mathbf{g}^T \mathbf{x} = h$ at exactly **one point of first contact**. This tangency point is the constrained optimum $\mathbf{x}_{\text{con}}^*$. Because the solution lies strictly on the boundary ($\mathbf{g}^T \mathbf{x}_{\text{con}}^* = h$), the constraint is said to be **active**.

3.4 The KKT Conditions: Geometric Force Balance

At the constrained optimum $\mathbf{x}_{\text{con}}^*$ on an active boundary, the gradient $\nabla f(\mathbf{x}_{\text{con}}^*)$ cannot point parallel to the boundary; if it did, one could move along the line and further reduce the cost.

Therefore, the contour ellipse must be **tangent** to the constraint boundary $\mathbf{g}^T \mathbf{x} = h$. Mathematically, this means the gradient of the objective must be collinear with the normal vector of the constraint:

$$\nabla f(\mathbf{x}^*) + \lambda \mathbf{g} = \mathbf{0} \iff -\nabla f(\mathbf{x}^*) = \lambda \mathbf{g} \quad (19)$$

where $\lambda \geq 0$ is the **Lagrange multiplier**.

Notice the profound physical intuition of Equation (19):

- The vector $-\nabla f(\mathbf{x}^*)$ is the **direction of steepest descent** (the direction the optimizer wants to roll down the bowl).
- Because $\mathbf{x}_{\text{unc}}^*$ is in the forbidden zone, $-\nabla f(\mathbf{x}^*)$ points outward into the infeasible region.
- The constraint boundary acts like a rigid mechanical wall. It exerts an inward normal reaction force $\lambda \mathbf{g}$ with magnitude $\lambda > 0$ that prevents the solution from entering the forbidden zone.

Mathematical Definition: Karush-Kuhn-Tucker (KKT) Optimality Conditions for QP

For the inequality-constrained quadratic program $\min \frac{1}{2} \mathbf{x}^T \mathbf{P} \mathbf{x} + \mathbf{q}^T \mathbf{x}$ subject to $\mathbf{G} \mathbf{x} \leq \mathbf{h}$, a point $\mathbf{x}^*$ and multiplier vector $\boldsymbol{\lambda}^*$ are globally optimal if and only if they satisfy:

$$\mathbf{P} \mathbf{x}^* + \mathbf{q} + \mathbf{G}^T \boldsymbol{\lambda}^* = \mathbf{0} \quad (\text{Stationarity: Force Balance}) \quad (20)$$

$$\mathbf{G} \mathbf{x}^* \leq \mathbf{h} \quad (\text{Primal Feasibility}) \quad (21)$$

$$\boldsymbol{\lambda}^* \geq \mathbf{0} \quad (\text{Dual Feasibility: Walls Only Push}) \quad (22)$$

$$\lambda_i^* (h_i - \mathbf{g}_i^T \mathbf{x}^*) = 0 \quad \forall i \quad (\text{Complementary Slackness}) \quad (23)$$

Step-by-Step Worked Example: Analytical Solution for Active 2D Constraint

Continuing our numerical example from Section 2, suppose we impose the constraint:

$$x_1 + 1.5x_2 \leq 2.2 \implies \mathbf{g} = \begin{bmatrix} 1.0 \\ 1.5 \end{bmatrix}, \quad h = 2.2$$

Step 1: Test Unconstrained Point:

$$\mathbf{g}^T \mathbf{x}_{\text{unc}}^* = (1.0)(1.143) + (1.5)(1.429) = 1.143 + 2.143 = 3.286 > 2.2$$

The constraint is violated! Therefore, it **must be active** at the optimum ($\lambda > 0$ and $\mathbf{g}^T \mathbf{x}^* = h$).

Step 2: Solve KKT System: From stationarity: $\mathbf{x}^* = \mathbf{x}_{\text{unc}}^* - \lambda \mathbf{P}^{-1} \mathbf{g}$. Substituting into the active boundary $\mathbf{g}^T \mathbf{x}^* = h$:

$$\mathbf{g}^T (\mathbf{x}_{\text{unc}}^* - \lambda \mathbf{P}^{-1} \mathbf{g}) = h \implies \lambda = \frac{\mathbf{g}^T \mathbf{x}_{\text{unc}}^* - h}{\mathbf{g}^T \mathbf{P}^{-1} \mathbf{g}}$$

Compute $\mathbf{P}^{-1} \mathbf{g}$:

$$\mathbf{P}^{-1} \mathbf{g} = \frac{1}{7} \begin{bmatrix} 2 & -1 \\ -1 & 4 \end{bmatrix} \begin{bmatrix} 1.0 \\ 1.5 \end{bmatrix} = \frac{1}{7} \begin{bmatrix} 0.5 \\ 5.0 \end{bmatrix} \approx \begin{bmatrix} 0.0714 \\ 0.7143 \end{bmatrix}$$

Compute denominator:

$$\mathbf{g}^T \mathbf{P}^{-1} \mathbf{g} = 1.0(0.0714) + 1.5(0.7143) = 0.0714 + 1.0714 = 1.1428$$

Compute λ :

$$\lambda = \frac{3.2857 - 2.2}{1.1428} = \frac{1.0857}{1.1428} \approx 0.9500 > 0 \quad (\text{Dual feasible!})$$

Step 3: Compute Constrained Optimum $\mathbf{x}_{\text{con}}^*$:

$$\mathbf{x}_{\text{con}}^* = \begin{bmatrix} 1.1429 \\ 1.4286 \end{bmatrix} - 0.9500 \begin{bmatrix} 0.0714 \\ 0.7143 \end{bmatrix} \approx \begin{bmatrix} 1.075 \\ 0.750 \end{bmatrix}$$

Check boundary: $1.075 + 1.5(0.750) = 1.075 + 1.125 = 2.200 = h$. Exact boundary satisfaction!

4 The General Quadratic Program and the CVXOPT Solver

4.1 Canonical Standard Formulation

When moving from 2D toy problems to multidimensional robotic systems, manual algebraic inversion and case checking becomes impossible. Instead, we express the problem in standard matrix form and invoke specialized numerical solvers.

Mathematical Definition: Canonical Primal Quadratic Program

The standard formulation of a general Quadratic Program is:

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & \frac{1}{2} \mathbf{x}^T \mathbf{P} \mathbf{x} + \mathbf{q}^T \mathbf{x} \\ \text{subject to} \quad & \mathbf{G} \mathbf{x} \leq \mathbf{h} \quad (m \text{ linear inequalities}) \\ & \mathbf{A} \mathbf{x} = \mathbf{b} \quad (p \text{ linear equalities}) \end{aligned} \tag{24}$$

where:

- $\mathbf{x} \in \mathbb{R}^n$ is the unknown decision vector.
- $\mathbf{P} \in \mathbb{S}_+^n$ is a symmetric positive semi-definite matrix ($\mathbf{P} \succeq 0$).
- $\mathbf{q} \in \mathbb{R}^n$ is the linear cost vector.
- $\mathbf{G} \in \mathbb{R}^{m \times n}$ and $\mathbf{h} \in \mathbb{R}^m$ define the polyhedral feasible inequality set.
- $\mathbf{A} \in \mathbb{R}^{p \times n}$ and $\mathbf{b} \in \mathbb{R}^p$ define affine equality constraints ($p < n$).

4.2 Overview of the CVXOPT Solver

CVXOPT (Python Software for Convex Optimization) is an established, high-performance open-source package developed by Lieven Vandenbergh and Martin Dahl. It solves general convex optimization problems, linear programs, and quadratic programs using **primal-dual interior-point algorithms**.

Key characteristics of CVXOPT:

- **Algorithmic Core:** Implements Mehrotra’s predictor-corrector primal-dual path-following algorithm.
- **Efficiency:** Written in C with BLAS/LAPACK linear algebra bindings, delivering sub-millisecond execution times.
- **Custom Matrix Class:** Uses its own dense and sparse representation (`cvxopt.matrix`), requiring double-precision floats (`float64`).

4.3 Solving QPs with `cvxopt.solvers.qp`

The primary function for solving Equation (24) in CVXOPT is:

```
1 from cvxopt import matrix, solvers
2
3 solution = solvers.qp(P, q, G, h, A=None, b=None)
```

4.3.1 Important Usage Guidelines

1. **Data Types:** All matrices and vectors **must** be created as `cvxopt.matrix` with floating-point values. Integer NumPy arrays will throw a `TypeError`.
2. **Array Layout:** When converting NumPy 2D arrays to `cvxopt.matrix`, set `dtype=np.float64`.
3. **Muting Output in Control Loops:** By default, CVXOPT prints iteration tables to stdout. In high-speed robotic control loops (50–100 Hz), printing creates massive I/O latency. Mute it unconditionally using:


```

1 solvers.options['show_progress'] = False
2

```

4. **Omission of Constraints:** If equality constraints are not present, omit A and b or set them to `None`.

5. **Parsing Results:** The return object is a dictionary containing:

- `sol['status']`: Convergence flag ('optimal', 'primal infeasible', etc.).
- `sol['x']`: Optimal primal decision vector $\mathbf{x}^*$ (`cvxopt.matrix`). Convert to NumPy with `np.array(sol['x']).flatten()`.
- `sol['primal objective']`: Scalar minimum cost value $f(\mathbf{x}^*)$.
- `sol['z']`: Optimal Lagrange multipliers $\boldsymbol{\lambda}^*$ for inequality constraints.

4.4 Self-Contained Python Example

The following complete, runnable Python script solves the exact 2D constrained problem derived in Section 3:

```

1 import numpy as np
2 from cvxopt import matrix, solvers
3
4 # 1. Mute solver console output for fast, clean execution
5 solvers.options['show_progress'] = False
6
7 # 2. Define Problem Data in NumPy (float64)
8 # Cost: 0.5 * x^T P x + q^T x
9 P_np = np.array([[4.0, 1.0],
10                  [1.0, 2.0]], dtype=np.float64)
11 q_np = np.array([-6.0, -4.0], dtype=np.float64)
12
13 # Constraint: g^T x <= h -> 1.0*x1 + 1.5*x2 <= 2.2
14 G_np = np.array([[1.0, 1.5]], dtype=np.float64)
15 h_np = np.array([2.2], dtype=np.float64)
16
17 # 3. Convert to CVXOPT matrices
18 P = matrix(P_np)
19 q = matrix(q_np)
20 G = matrix(G_np)
21 h = matrix(h_np)
22
23 # 4. Solve the Quadratic Program
24 solution = solvers.qp(P, q, G, h)
25
26 # 5. Extract and Validate Results
27 status = solution['status']
28 x_star = np.array(solution['x']).flatten()
29 cost = solution['primal objective'] + 10.0 # adding constant r = 10
30 lambda_star = np.array(solution['z']).flatten()
31
32 print(f"Solver Status:      {status}")
33 print(f"Optimal x*:          [{x_star[0]:.4f}, {x_star[1]:.4f}]^T")
34 print(f"Lagrange Multiplier: {lambda_star[0]:.4f}")
35 print(f"Minimum Cost:         {cost:.4f}")

```

Listing 1: Solving the 2D Constrained QP with CVXOPT

Execution Output:

```

Solver Status:      optimal
Optimal x*:        [1.0750, 0.7500]^T
Lagrange Multiplier: 0.9500
Minimum Cost:       4.2300

```

The numerical solution computed by CVXOPT matches our analytical KKT derivation to the fourth decimal place!

5 Robotics Application: Multi-Obstacle Control Barrier Functions

We now demonstrate the full practical utility of Quadratic Programming by applying it to an advanced problem in aerial robotics: **Multi-Obstacle Control Barrier Function (CBF) Safety Filtering** for autonomous quadrotors.

5.1 System Architecture and Single-Integrator Kinematics

Modern quadrotor autonomy stacks frequently employ a hierarchical two-tier control architecture:

- **Tier 1 (High-Level Kinematic Guidance):** A position planner computes desired velocity commands $\mathbf{v}_{\text{des}} \in \mathbb{R}^3$ at 50 Hz.
- **Tier 2 (Lower-Level Dynamics):** An inner-loop attitude controller stabilizes roll, pitch, and yaw angles to track $\mathbf{v}_{\text{des}}$, allocating thrusts across the four brushless motors.

At the high-level guidance tier, the translational kinematics of the quadrotor position $\mathbf{p} = [x, y, z]^T \in \mathbb{R}^3$ are modeled as a **single-integrator**:

$$\dot{\mathbf{p}} = \mathbf{v} \quad (25)$$

where the commanded velocity $\mathbf{v} \in \mathbb{R}^3$ acts directly as the control input.

5.2 Nominal Guidance: Defining $\mathbf{v}_{\text{des}}$ via a Position PID Controller

To navigate toward a target goal waypoint $\mathbf{p}_{\text{goal}} \in \mathbb{R}^3$, the high-level guidance tier relies on a nominal controller. In modern robotic flight stacks, this nominal velocity command $\mathbf{v}_{\text{des}}$ is generated by a standard 3-axis **Position Proportional-Integral-Derivative (PID)** controller.

5.2.1 Continuous-Time Position PID Formulation

Let $\mathbf{e}(t) \in \mathbb{R}^3$ denote the instantaneous 3D position error between the target waypoint and the quadrotor's current position:

$$\mathbf{e}(t) = \mathbf{p}_{\text{goal}}(t) - \mathbf{p}(t) \quad (26)$$

The continuous-time nominal velocity command $\mathbf{v}_{\text{PID}}(t)$ is given by:

$$\mathbf{v}_{\text{PID}}(t) = \mathbf{K}_p \mathbf{e}(t) + \mathbf{K}_i \int_0^t \mathbf{e}(\tau) d\tau + \mathbf{K}_d \frac{d\mathbf{e}(t)}{dt} \quad (27)$$

where $\mathbf{K}_p = \text{diag}(k_p, k_p, k_p) \succ 0$, $\mathbf{K}_i = \text{diag}(k_i, k_i, k_i) \succ 0$, and $\mathbf{K}_d = \text{diag}(k_d, k_d, k_d) \succ 0$ are positive definite diagonal gain matrices.

Each term serves a distinct physical role:

- **Proportional Term ($\mathbf{K}_p \mathbf{e}$):** Acts as a virtual spring pulling the quadrotor toward the goal with an intensity proportional to positional offset.

- **Integral Term** ($\mathbf{K}_i \int \mathbf{e} d\tau$): Accumulates persistent offsets, eliminating steady-state droop caused by constant unmodeled disturbances (such as aerodynamic rotor drag or steady wind gusts).
- **Derivative Term** ($\mathbf{K}_d \dot{\mathbf{e}}$): Provides viscous damping proportional to relative approach velocity. For a stationary waypoint ($\dot{\mathbf{p}}_{\text{goal}} = \mathbf{0}$), $\frac{d\mathbf{e}}{dt} = -\dot{\mathbf{p}} = -\mathbf{v}$, acting as velocity feedback that suppresses setpoint overshoot and oscillations.

5.2.2 Integral Anti-Windup Clamping and Speed Saturation

In real flight systems, two non-linearities augment the ideal linear PID law:

1. **Integral Anti-Windup**: When the quadrotor is detained by an obstacle or during large setpoint steps, naive integration causes the integral term to grow unbounded (integrator windup). When the obstacle is cleared, the massive stored integral produces violent velocity overshoot. To prevent this, the integrator state $\mathbf{e}_{\text{int}}(t)$ is bounded by saturation thresholds:

$$\mathbf{e}_{\text{int}}(t) = \text{clamp} \left(\int_0^t \mathbf{e}(\tau) d\tau, -\mathbf{e}_{\text{max}}^{\text{int}}, \mathbf{e}_{\text{max}}^{\text{int}} \right) \quad (28)$$

2. **Physical Speed Saturation**: Physical quadrotor aerodynamics and motor limits impose a maximum safe airspeed v_{max} (e.g., 1.8 m/s). The raw PID output is normalized if it exceeds v_{max} :

$$\mathbf{v}_{\text{des}}(t) = \text{sat}_{v_{\text{max}}}(\mathbf{v}_{\text{PID}}(t)) = \begin{cases} \mathbf{v}_{\text{PID}}(t) & \text{if } \|\mathbf{v}_{\text{PID}}(t)\| \leq v_{\text{max}} \\ v_{\text{max}} \frac{\mathbf{v}_{\text{PID}}(t)}{\|\mathbf{v}_{\text{PID}}(t)\|} & \text{if } \|\mathbf{v}_{\text{PID}}(t)\| > v_{\text{max}} \end{cases} \quad (29)$$

5.2.3 Discrete-Time Implementation (50 Hz Control Loop)

In digital implementation with sampling period $\Delta t = 0.02$ s (50 Hz), at control step k :

$$\mathbf{e}_k = \mathbf{p}_{\text{goal}} - \mathbf{p}_k \quad (30)$$

$$\mathbf{e}_{\text{int},k} = \text{clip}(\mathbf{e}_{\text{int},k-1} + \mathbf{e}_k \Delta t, -\mathbf{e}_{\text{max}}^{\text{int}}, \mathbf{e}_{\text{max}}^{\text{int}}) \quad (31)$$

$$\dot{\mathbf{e}}_k = \frac{\mathbf{e}_k - \mathbf{e}_{k-1}}{\Delta t} \quad (32)$$

$$\mathbf{v}_{\text{PID},k} = k_p \mathbf{e}_k + k_i \mathbf{e}_{\text{int},k} + k_d \dot{\mathbf{e}}_k \quad (33)$$

$$\mathbf{v}_{\text{des},k} = \begin{cases} \mathbf{v}_{\text{PID},k} & \text{if } \|\mathbf{v}_{\text{PID},k}\| \leq v_{\text{max}} \\ v_{\text{max}} \frac{\mathbf{v}_{\text{PID},k}}{\|\mathbf{v}_{\text{PID},k}\|} & \text{otherwise} \end{cases} \quad (34)$$

5.2.4 Why the PID Command Must Be Filtered by a CBF

While the position PID controller is exceptionally effective at driving the quadrotor to $\mathbf{p}_{\text{goal}}$, it is **fundamentally obstacle-blind**. If an obstacle is located directly between $\mathbf{p}$ and $\mathbf{p}_{\text{goal}}$, the error vector $\mathbf{e}$ points directly through the obstacle center, commanding the quadcopter to ram straight into the obstacle at full speed!

The Control Barrier Function acts as an **instantaneous safety filter** inserted directly between the PID guidance loop and the lower-level attitude tracking loop:

- When the nominal PID command $\mathbf{v}_{\text{des}}$ is safe (moving parallel to or away from obstacles), the CBF safety filter leaves it completely untouched: $\mathbf{v}^* = \mathbf{v}_{\text{des}}$.
- When $\mathbf{v}_{\text{des}}$ would penetrate an obstacle's safety buffer, the CBF Quadratic Program projects $\mathbf{v}_{\text{des}}$ minimally onto the boundary of the safe velocity set:

$$\mathbf{v}^* = \arg \min_{\mathbf{v} \in \mathcal{K}(\mathbf{p})} \frac{1}{2} \|\mathbf{v} - \mathbf{v}_{\text{des}}^{\text{PID}}\|^2 \quad (35)$$

5.3 Quantifying Safety: The Barrier Margin $h_i(\mathbf{p})$

Consider an environment populated by M static spherical obstacles. Each obstacle $i \in \{1, \dots, M\}$ is centered at $\mathbf{p}_{\text{obs},i} \in \mathbb{R}^3$ with a safety buffer radius $D_{\text{obs},i}$:

$$D_{\text{obs},i} = R_{\text{obs},i} + R_{\text{drone}} + d_{\text{buffer}} \quad (36)$$

where $R_{\text{obs},i}$ is the physical radius of the obstacle, $R_{\text{drone}} \approx 0.17$ m accounts for the spinning propeller blade span, and d_{buffer} is a margin of conservatism.

For each obstacle, we define a scalar **safety barrier function** $h_i(\mathbf{p}) : \mathbb{R}^3 \rightarrow \mathbb{R}$:

$$h_i(\mathbf{p}) = \|\mathbf{p} - \mathbf{p}_{\text{obs},i}\| - D_{\text{obs},i} \quad (37)$$

The sign of $h_i(\mathbf{p})$ establishes three distinct regions:

- **Safe Set** ($\mathcal{S}_i$): $\mathcal{S}_i = \{\mathbf{p} \in \mathbb{R}^3 \mid h_i(\mathbf{p}) \geq 0\}$ (outside or on the safety boundary).
- **Boundary Surface** ($\partial\mathcal{S}_i$): $\partial\mathcal{S}_i = \{\mathbf{p} \in \mathbb{R}^3 \mid h_i(\mathbf{p}) = 0\}$ (the safety sphere shell).
- **Forbidden Set** ($\text{Int}(\mathcal{S}_i^c)$): $\{\mathbf{p} \in \mathbb{R}^3 \mid h_i(\mathbf{p}) < 0\}$ (safety buffer penetrated).

The outward gradient $\nabla h_i(\mathbf{p})$ is the unit normal vector pointing radially away from obstacle i :

$$\nabla h_i(\mathbf{p}) = \frac{\mathbf{p} - \mathbf{p}_{\text{obs},i}}{\|\mathbf{p} - \mathbf{p}_{\text{obs},i}\|}, \quad \|\nabla h_i(\mathbf{p})\| = 1 \quad (38)$$

5.4 Forward Set Invariance and Nagumo's Condition

As the drone flies with velocity $\mathbf{v}$, the rate of change of the safety margin is:

$$\dot{h}_i(\mathbf{p}, \mathbf{v}) = \frac{\partial h_i}{\partial \mathbf{p}} \dot{\mathbf{p}} = \nabla h_i(\mathbf{p})^T \mathbf{v} \quad (39)$$

Mathematical Definition: Forward Invariance Condition (Nagumo's Theorem / Ames et al.)

The joint safe set $\mathcal{S} = \bigcap_{i=1}^M \mathcal{S}_i$ is **forward invariant** (i.e., if $\mathbf{p}(0) \in \mathcal{S}$, then $\mathbf{p}(t) \in \mathcal{S}$ for all $t \geq 0$) if the commanded velocity $\mathbf{v}$ satisfies:

$$\dot{h}_i(\mathbf{p}, \mathbf{v}) \geq -\alpha_i h_i(\mathbf{p}) \iff -\nabla h_i(\mathbf{p})^T \mathbf{v} \leq \alpha_i h_i(\mathbf{p}), \quad \forall i \in \{1, \dots, M\} \quad (40)$$

where $\alpha_i > 0$ is a tunable class- $\mathcal{K}$ scalar barrier gain.

Intuition & Geometric Insight: Physical Meaning of the Barrier Gain α

Equation (40) acts as an instantaneous, distance-dependent directional speed governor:

- When far away ($h_i \gg 0$), the right-hand side $\alpha_i h_i$ is large and positive. The inward velocity $-\nabla h_i^T \mathbf{v}$ can be large; the drone is allowed to fly toward the obstacle at full speed.
- As the drone nears the boundary ($h_i \rightarrow 0$), the permissible inward velocity throttles smoothly to zero.
- At the boundary ($h_i = 0$), the condition enforces $\nabla h_i^T \mathbf{v} \geq 0$: the drone cannot move toward the obstacle at all, but may move freely tangentially or away from it.
- **Tuning α** : Smaller α (e.g., 0.8–1.0) enforces earlier, gentler braking, which is essential on physical quadcopters to compensate for attitude tilt latency. Larger α allows later, sharper intervention.

5.5 Why Multiple Obstacles Demand a QP Solver

In a **single-obstacle** environment ($M = 1$), the barrier condition is a single linear half-space in $\mathbb{R}^3$. Because $\|\nabla h\| = 1$, the optimal safe velocity admits an exact, closed-form orthogonal projection formula:

$$\mathbf{v}^* = \mathbf{v}_{\text{des}} - \min(0, \nabla h^T \mathbf{v}_{\text{des}} + \alpha h) \nabla h \quad (41)$$

However, in realistic autonomous operations, the drone navigates cluttered fields with **multiple obstacles** ($M \geq 2$).

Each obstacle i enforces an independent half-space constraint:

$$-\nabla h_i(\mathbf{p})^T \mathbf{v} \leq \alpha_i h_i(\mathbf{p}) \quad (42)$$

The intersection of these M half-spaces forms a **convex polyhedral cone** in velocity space:

$$\mathcal{K}(\mathbf{p}) = \{\mathbf{v} \in \mathbb{R}^3 \mid -\nabla h_i(\mathbf{p})^T \mathbf{v} \leq \alpha_i h_i(\mathbf{p}), \quad \forall i = 1, \dots, M\} \quad (43)$$

When the nominal command $\mathbf{v}_{\text{des}}$ violates multiple barrier constraints simultaneously (e.g., when passing between two close obstacles):

- Applying independent scalar projections sequentially fails, because projecting away from Obstacle 1 can push the velocity into the forbidden half-space of Obstacle 2!
- The optimal safe command lies at the intersection of active constraint hyperplanes (e.g., a face or vertex of the polytope $\mathcal{K}(\mathbf{p})$).

Finding the minimally invasive safe velocity $\mathbf{v}^*$ that stays closest to $\mathbf{v}_{\text{des}}$ in the Euclidean sense requires solving a **Quadratic Program** online at every control cycle (50 Hz).

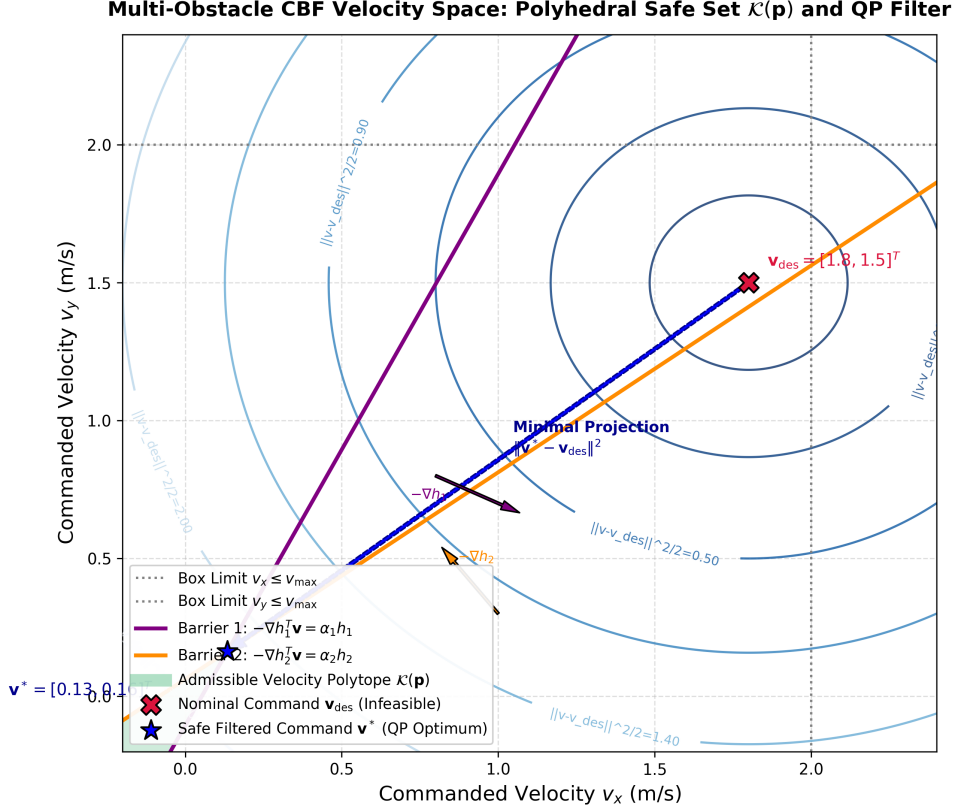


Figure 2: **Multi-Obstacle CBF Velocity Space Geometry.** Two obstacles impose simultaneous linear barrier boundaries (purple and orange lines). The intersection of these half-spaces with physical speed limits defines the green admissible safe velocity polytope $\mathcal{K}(\mathbf{p})$. When nominal velocity $\mathbf{v}_{\text{des}}$ (red X) violates both barriers, the QP safety filter projects it minimally onto the boundary of $\mathcal{K}(\mathbf{p})$, producing the globally optimal safe velocity $\mathbf{v}^*$ (blue star).

5.6 Formulating the Multi-Obstacle CBF-QP

The minimally invasive safety filter is formulated as the following instantaneous Quadratic Program:

$$\begin{aligned} \mathbf{v}^*(\mathbf{p}) = \arg \min_{\mathbf{v} \in \mathbb{R}^3} \quad & \frac{1}{2} \|\mathbf{v} - \mathbf{v}_{\text{des}}\|^2 \\ \text{subject to} \quad & -\nabla h_i(\mathbf{p})^T \mathbf{v} \leq \alpha_i h_i(\mathbf{p}), \quad \forall i \in \{1, \dots, M\} \\ & -v_{\max} \leq v_j \leq v_{\max}, \quad \forall j \in \{x, y, z\} \end{aligned} \quad (44)$$

5.7 Mapping to CVXOPT Standard Matrices

To solve Equation (44) with CVXOPT, we expand the objective function:

$$\frac{1}{2} \|\mathbf{v} - \mathbf{v}_{\text{des}}\|^2 = \frac{1}{2} \mathbf{v}^T \mathbf{I}_{3 \times 3} \mathbf{v} - \mathbf{v}_{\text{des}}^T \mathbf{v} + \frac{1}{2} \|\mathbf{v}_{\text{des}}\|^2 \quad (45)$$

Comparing with $\frac{1}{2} \mathbf{v}^T \mathbf{P} \mathbf{v} + \mathbf{q}^T \mathbf{v}$, we identify:

$$\mathbf{P} = \mathbf{I}_{3 \times 3} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{q} = -\mathbf{v}_{\text{des}} = \begin{bmatrix} -v_{\text{des},x} \\ -v_{\text{des},y} \\ -v_{\text{des},z} \end{bmatrix} \quad (46)$$

For M obstacles and 3D box velocity limits ($|v_j| \leq v_{\max}$), we construct $\mathbf{G} \in \mathbb{R}^{(M+6) \times 3}$ and $\mathbf{h} \in \mathbb{R}^{M+6}$:

$$\mathbf{G} = \begin{bmatrix} -\nabla h_1(\mathbf{p})^T \\ \vdots \\ -\nabla h_M(\mathbf{p})^T \\ \mathbf{I}_{3 \times 3} \\ -\mathbf{I}_{3 \times 3} \end{bmatrix}, \quad \mathbf{h} = \begin{bmatrix} \alpha_1 h_1(\mathbf{p}) \\ \vdots \\ \alpha_M h_M(\mathbf{p}) \\ v_{\max} \mathbf{1}_3 \\ v_{\max} \mathbf{1}_3 \end{bmatrix} \quad (47)$$

5.8 Python Implementation: MultiObstacleCBFSafetyFilter

Below is a complete, production-ready implementation of the multi-obstacle CBF filter using CVXOPT:

```

1 import time
2 import numpy as np
3 import cvxopt
4 from cvxopt import matrix, solvers
5
6 # Mute solver stdout for real-time operation in 50 Hz control loops
7 solvers.options['show_progress'] = False
8
9 class MultiObstacleCBFSafetyFilter:
10     """
11     Online Multi-Obstacle CBF Safety Filter using CVXOPT.
12     Solves at 50 Hz:
13         min_v 0.5 * ||v - v_des||^2
14         s.t.  -nabla h_i(p)^T v <= alpha * h_i(p),    forall i in {1..M}
15              -v_max <= v_j <= v_max,                  j in {x, y, z}
16     """
17     def __init__(self, obstacles, D_obs=0.55, alpha=0.9, v_max=1.8):
18         self.obstacles = [np.array(obs, dtype=np.float32) for obs in obstacles]
19         self.D_obs = [D_obs]*len(obstacles) if isinstance(D_obs, (int, float))
20         else list(D_obs)
21         self.alpha = alpha
22         self.v_max = v_max
23         # P matrix is constant 3x3 identity
24         self.P_mat = matrix(np.eye(3, dtype=np.float64))
25
26     def update_obstacles(self, obstacles):
27         """Update obstacle coordinates dynamically at runtime."""
28         self.obstacles = [np.array(obs, dtype=np.float32) for obs in obstacles]
29
30     def filter(self, v_des, current_pos):
31         current_pos = np.array(current_pos, dtype=np.float32)
32         v_nom = v_des.copy()
33
34         h_vals = []
35         G_rows = []
36         h_rows = []
37
38         # 1. Build barrier constraints for each obstacle
39         for i, p_obs in enumerate(self.obstacles):
40             diff = current_pos - p_obs
41             dist = float(np.linalg.norm(diff))
42             normal = diff / dist if dist > 1e-4 else np.array([1.0, 0.0, 0.0],
43 dtype=np.float32)
44             h_i = dist - self.D_obs[i]
45             h_vals.append(h_i)
46
47             # Barrier condition: -normal^T v <= alpha * h_i
48             G_rows.append(-normal.astype(np.float64))
49             h_rows.append(float(self.alpha * h_i))

```

```

48
49     # 2. Fast check: is nominal command already feasible?
50     is_safe = True
51     for row, bound in zip(G_rows, h_rows):
52         if np.dot(row, v_nom) > bound + 1e-5:
53             is_safe = False
54             break
55
56     if is_safe and np.all(np.abs(v_nom) <= self.v_max):
57         return v_nom.astype(np.float32), h_vals, 0.0
58
59     # 3. Stack box velocity bounds [-v_max, v_max]^3
60     G_all = G_rows.copy()
61     h_all = h_rows.copy()
62     for j in range(3):
63         unit = np.zeros(3, dtype=np.float64)
64         unit[j] = 1.0
65         G_all.append(unit)
66         h_all.append(float(self.v_max))
67         G_all.append(-unit)
68         h_all.append(float(self.v_max))
69
70     # 4. Convert to CVXOPT matrices
71     q_mat = matrix(-v_nom.astype(np.float64))
72     G_mat = matrix(np.array(G_all, dtype=np.float64))
73     h_mat = matrix(np.array(h_all, dtype=np.float64))
74
75     # 5. Solve QP online
76     t0 = time.perf_counter()
77     try:
78         sol = solvers.qp(self.P_mat, q_mat, G_mat, h_mat)
79         solve_time_ms = (time.perf_counter() - t0) * 1000.0
80         if sol['status'] == 'optimal':
81             v_star = np.array(sol['x']).flatten().astype(np.float32)
82         else:
83             v_star = v_nom # Fallback
84     except Exception:
85         solve_time_ms = 0.0
86         v_star = v_nom
87
88     return v_star, h_vals, solve_time_ms

```

Listing 2: Production-Grade Multi-Obstacle CBF Safety Filter Class

5.9 Numerical Case Study

To evaluate the filter, we simulate a quadrotor at $\mathbf{p} = [0.55, 0.45, 0.85]^T$ commanding an aggressive nominal velocity $\mathbf{v}_{\text{des}} = [1.2, 1.0, 0.5]^T$ m/s directly toward three obstacles:

- Obstacle 1: $\mathbf{p}_{\text{obs},1} = [0.50, 0.35, 0.85]^T$, $D_{\text{obs}} = 0.55$ m. Distance = 0.112 m $\implies h_1 = -0.438$ m (in boundary buffer).
- Obstacle 2: $\mathbf{p}_{\text{obs},2} = [0.65, 0.75, 1.15]^T$, $D_{\text{obs}} = 0.55$ m. Distance = 0.442 m $\implies h_2 = -0.108$ m.
- Obstacle 3: $\mathbf{p}_{\text{obs},3} = [1.05, 1.00, 1.40]^T$, $D_{\text{obs}} = 0.55$ m. Distance = 0.925 m $\implies h_3 = +0.375$ m.

Feeding this scenario into our `MultiObstacleCBFSafetyFilter` yields:

- **Nominal Velocity:** $\mathbf{v}_{\text{des}} = [1.200, 1.000, 0.500]^T$ m/s (speed = 1.640 m/s).

- **Filtered Safe Velocity:** $\mathbf{v}^* = [0.876, 0.029, -0.471]^T$ m/s (speed = 0.996 m/s).
- **Solve Time:** 0.45 ms ($< 2.5\%$ of the 20 ms budget at 50 Hz).
- **Verification:** The safety filter sharply reduces the lateral velocity v_y and redirects v_z downward to steer safely around the obstacle field, guaranteeing $-\nabla h_i^T \mathbf{v}^* \leq \alpha h_i$ across all obstacles.

6 Summary and Practical Takeaways

1. **Unconstrained QP is Linear Algebra:** The unconstrained minimization of a strictly convex quadratic function $\frac{1}{2}\mathbf{x}^T \mathbf{P} \mathbf{x} + \mathbf{q}^T \mathbf{x}$ reduces directly to solving the linear system $\mathbf{P} \mathbf{x} = -\mathbf{q}$, giving $\mathbf{x}_{\text{unc}}^* = -\mathbf{P}^{-1} \mathbf{q}$.
2. **Constraints Reshape Geometry via Level Sets:** Linear constraints truncate the decision space into a convex polyhedron. Inactive constraints exert no influence ($\lambda = 0$). Active constraints balance the steepest descent direction $-\nabla f(\mathbf{x}^*)$ with normal reaction forces $\lambda_i \mathbf{g}_i$ ($\lambda_i > 0$), formalized by the KKT stationarity condition $\nabla f(\mathbf{x}^*) + \mathbf{G}^T \boldsymbol{\lambda}^* = \mathbf{0}$.
3. **CVXOPT Enables Real-Time Execution:** Using primal-dual interior-point algorithms, CVXOPT solves multi-constraint QPs in under 0.5 ms, making it an ideal choice for high-frequency robotic control loops.
4. **Multi-Obstacle CBF as a Canonical QP:** While single-obstacle CBFs admit closed-form scalar projections, multi-obstacle environments generate intersecting half-space constraints forming a convex polytope $\mathcal{K}(\mathbf{p})$. Formulating safety filtering as a QP guarantees minimal modification to nominal mission guidance while provably ensuring collision avoidance.

References

- [1] S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.
- [2] J. Nocedal and S. J. Wright, *Numerical Optimization*, Springer Science & Business Media, 2nd ed., 2006.
- [3] M. S. Andersen, J. Dahl, and L. Vandenberghe, “CVXOPT: A Python package for convex optimization,” v1.1.6, available at <http://cvxopt.org/>, 2013.
- [4] A. D. Ames, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs with application to adaptive cruise control,” in *53rd IEEE Conference on Decision and Control (CDC)*, pp. 6271–6278, 2014.
- [5] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [6] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” in *18th European Control Conference (ECC)*, pp. 3420–3431, 2019.